Data Structure Complexity Cheat Sheet

Notation: n = number of elements, L = key/string length, V = vertices, E = edges, k = result size / hash count, B = B-tree branching factor. Best and worst case are separate columns; average case is mentioned in Notes where it differs a lot (hash, BST, skip list). “amortized” = averaged over a sequence of operations (one op can be slower, e.g. a resize).

---

1. Time and space complexity matrix

How to read it:

- Insert / Delete = add or remove one element the normal way for that structure (stack push / pop, queue enqueue / dequeue, heap extract top, hash and tree by key).
- Search = find by value or key. Update = change the value of an existing element found by key / value (a key change in an ordered structure = delete + insert).
- n/a = not supported. `*` = amortized.

Data structure	Insert best	Insert worst	Search best	Search worst	Delete best	Delete worst	Update best	Update worst	Space	Notes
Linear										
Array (fixed)	O(1)	O(n)	O(1)	O(n)	O(1)	O(n)	O(1)	O(n)	O(n)	End is O(1); front / middle shifts O(n). Index read / update O(1)
Dynamic array	O(1)*	O(n)	O(1)	O(n)	O(1)	O(n)	O(1)	O(n)	O(n)	Append amortized O(1), resize O(n). Index read / update O(1)
Sorted array	O(1)	O(n)	O(1)	O(log n)	O(1)	O(n)	O(1)	O(n)	O(n)	Binary search O(log n); insert / delete shift O(n)
Singly linked list	O(1)	O(n)	O(1)	O(n)	O(1)	O(n)	O(1)	O(n)	O(n)	Head O(1); tail without tail pointer, or by value O(n)
Doubly linked list	O(1)	O(n)	O(1)	O(n)	O(1)	O(n)	O(1)	O(n)	O(n)	O(1) for any node already held; O(n) to find by value
Stack	O(1)	O(1)	O(1)	O(n)	O(1)	O(1)	O(1)	O(n)	O(n)	Push / pop / peek top O(1); arbitrary element O(n)
Queue	O(1)	O(1)	O(1)	O(n)	O(1)	O(1)	O(1)	O(n)	O(n)	Enqueue / dequeue / peek front O(1)
Deque	O(1)	O(1)	O(1)	O(n)	O(1)	O(1)	O(1)	O(n)	O(n)	O(1) at both ends; middle O(n)
Hash based										
Hash map / hash set	O(1)	O(n)	O(1)	O(n)	O(1)	O(n)	O(1)	O(n)	O(n)	Avg O(1). Worst O(n) on collisions or resize (O(log n) with Java 8+ tree buckets)
Bloom filter	O(k)	O(k)	O(k)	O(k)	n/a	n/a	n/a	n/a	O(m) bits	False positives possible, no false negatives; counting variant can delete
LRU cache (map + doubly linked list)	O(1)	O(1)	O(1)	O(1)	O(1)	O(1)	O(1)	O(1)	O(capacity)	get / put / evict all O(1)
Trees										
BST (unbalanced)	O(1)	O(n)	O(1)	O(n)	O(1)	O(n)	O(1)	O(n)	O(n)	Avg O(log n); worst O(n) on sorted input (skewed tree)
Balanced BST (AVL, Red-Black, TreeMap)	O(log n)	O(log n)	O(1)	O(log n)	O(log n)	O(log n)	O(1)	O(log n)	O(n)	Sorted iteration O(n); floor / ceiling / range O(log n)
Binary heap	O(1)	O(log n)	O(1)	O(n)	O(log n)	O(log n)	O(1)	O(n)	O(n)	Peek top O(1); build heap O(n); update key with known position O(log n)
Trie	O(L)	O(L)	O(1)	O(L)	O(1)	O(L)	O(1)	O(L)	O(N * L)	L = key length, independent of n; best case = first char missing
B-Tree / B+ Tree	O(log n)	O(log n)	O(1)	O(log n)	O(log n)	O(log n)	O(1)	O(log n)	O(n)	O(log_B n) disk reads; fast range scans (B+)
Skip list	O(log n)	O(n)	O(1)	O(n)	O(log n)	O(n)	O(1)	O(n)	O(n)	Avg O(log n); worst O(n) is very unlikely
Segment tree	n/a	n/a	O(log n)	O(log n)	n/a	n/a	O(log n)	O(log n)	O(n)	Search = range query; update = point update; build O(n)
Fenwick tree (BIT)	n/a	n/a	O(log n)	O(log n)	n/a	n/a	O(log n)	O(log n)	O(n)	Search = prefix query; update = point update; build O(n)

Graph and disjoint set

Data structure	Add vertex	Add edge	Remove edge	Edge exists?	Update edge weight	Remove vertex	Space
Adjacency list	O(1)	O(1)	O(1) / O(deg)	O(1) / O(deg)	O(1) / O(deg)	O(V + E)	O(V + E)
Adjacency matrix	O(V^2) resize	O(1)	O(1)	O(1)	O(1)	O(V^2)	O(V^2)
Edge list	O(1)	O(1)	O(E)	O(E)	O(E)	O(E)	O(E)

Data structure	Union	Find	Notes	Space
Union-Find (disjoint set), with path compression and union by rank	O(1) / O(alpha(n)) amortized	O(1) / O(alpha(n)) amortized	alpha(n) is under 5 for any practical n. Without the two optimizations the worst case is O(n). No delete.	O(n)

---

2. Examples of each

Data structure	Typical uses (interview and real life)	Java	Python	Go
Array	Fixed lookup tables, matrix / grid problems, DP tables, two pointers, sliding window, prefix sums	`int[]`	`array`, `list` used as fixed	`[N]int`
Dynamic array	Default list of things: results lists, collecting input, sorting, binary search on sorted data	`ArrayList`	`list`	slice `[]T`
Sorted array	Read-mostly lookup tables, binary search, merge intervals, `bisect` for rank / floor / ceiling	`Arrays.binarySearch`	`bisect`	`sort.Search`
Singly linked list	Reverse a list, detect cycle, merge sorted lists, adjacency chains in hash buckets	custom `Node`	custom	`container/list` (doubly) or custom
Doubly linked list	LRU cache, browser back / forward history, undo / redo, O(1) removal of a held node	`LinkedList`	custom	`container/list`
Stack	Parentheses matching, DFS (iterative), expression evaluation, undo, monotonic stack (next greater element), call stack	`ArrayDeque` (prefer over `Stack`)	`list` (`append`, `pop`)	slice
Queue	BFS, level-order traversal, task scheduling, producer / consumer, rate limiter windows	`ArrayDeque`, `LinkedList`	`collections.deque`	slice or channel
Deque	Sliding window maximum (monotonic deque), palindrome check, work stealing	`ArrayDeque`	`collections.deque`	slice or `container/list`
Hash map	Two-sum, frequency counts, caching / memoization, dedup, group anagrams, indexing by id	`HashMap`	`dict`	`map[K]V`
Hash set	Visited set in BFS / DFS, duplicate detection, membership checks	`HashSet`	`set`	`map[K]struct{}`
Bloom filter	“Probably seen” check before an expensive lookup (cache, DB, crawler URL dedup, spell check)	Guava `BloomFilter`	`pybloom`	`bits-and-blooms/bloom`
LRU cache	Page / object caches, CDN edge cache	`LinkedHashMap` (access order)	`OrderedDict`, `functools.lru_cache`	`container/list` + map
BST (unbalanced)	Teaching, validate BST, LCA, kth smallest; production use should be balanced	custom	custom	custom
Balanced BST	Ordered map / set: floor / ceiling, range queries, sorted iteration, leaderboard, calendar booking	`TreeMap`, `TreeSet`	`sortedcontainers.SortedDict`	no built-in; third-party
Binary heap / priority queue	Top-K, Dijkstra, merge K sorted lists, median of a stream (two heaps), task scheduler, event simulation	`PriorityQueue`	`heapq`	`container/heap`
Trie	Autocomplete, prefix search, spell check, IP routing (longest prefix), word search on grid	custom	custom	custom
B-Tree / B+ Tree	Database indexes (MySQL InnoDB, PostgreSQL), file system metadata	n/a (DB engine)	n/a	n/a
Skip list	Redis sorted sets, in-memory ordered maps with easy concurrency	`ConcurrentSkipListMap`	n/a	n/a
Segment tree	Range sum / min / max with point or range updates, e.g. “sum of [l, r] with updates”	custom	custom	custom

Data structure	Typical uses (interview and real life)	Java	Python	Go
Fenwick tree	Prefix sums with updates, counting inversions, rank queries	custom	custom	custom
Graph (adjacency list)	Sparse graphs: social networks, maps, dependency graphs, topological sort, shortest path	`Map<Integer, List<Integer>>`	`defaultdict(list)`	`map[int][]int`
Graph (adjacency matrix)	Dense graphs, small V, Floyd-Warshall, constant-time “is there an edge”	`int[][]`	list of lists	`[][]int`
Union-Find	Connected components, Kruskal MST, cycle detection in undirected graph, account merging, number of islands II	custom	custom	custom

---

3. Read vs write vs search: which one to use

Definitions: - Read = fetch by position or known handle (index, top, front, pointer). - Search = find by value or key (or by range / prefix). - Write = insert, delete, update.

By workload

Your workload	Best choice	Why	Avoid
Read by index, rarely modified	Array / dynamic array	O(1) index, best cache locality	linked list (O(n) index)
Append heavy, read by index	Dynamic array	O(1) amortized append, O(1) read	linked list (pointer overhead)
Search by exact key, any write rate	Hash map / hash set	O(1) avg search, insert, delete	tree, unless you need order
Search by key and need sorted order, floor / ceiling, or range	Balanced BST (TreeMap) or B+ tree	O(log n) for everything plus ordered iteration	hash map (no order)
Search on read-only (static) data	Sorted array + binary search	O(log n), compact, cache friendly, no pointers	rebuilding a tree
Heavy insert / delete at the ends	Deque / linked list	O(1) at both ends	array (O(n) front shifts)
Insert / delete in the middle with a node handle already held	Doubly linked list	O(1) splice	array (O(n) shift)
Always need the min or max	Heap / priority queue	O(1) peek, O(log n) push / pop	sorted array (O(n) insert)
Prefix queries / autocomplete	Trie	O(L), independent of n	scanning a hash map of strings
Range sums or min / max with updates	Segment tree / Fenwick tree	O(log n) both	prefix sum array (O(n) update)
Range sums, no updates	Prefix sum array	O(1) query, O(n) build	segment tree (overkill)
“Is it probably in the set?” with tight memory	Bloom filter	tiny memory, O(k)	big hash set
Connectivity / grouping queries	Union-Find	near O(1)	BFS on every query
Cache with eviction	Hash map + doubly linked list (LRU)	O(1) get and put	plain map (no eviction)
Relationships, sparse	Adjacency list	O(V + E) space	matrix (O(V^2))
Relationships, dense or O(1) edge check needed	Adjacency matrix	O(1) edge lookup	list (O(deg) lookup)
Very large data on disk, read-heavy	B+ tree	few disk reads, range scans	hash index if you need ranges
Very large data on disk, write-heavy	LSM tree (RocksDB, Cassandra)	sequential writes, batched compaction	B+ tree (random writes)

Quick comparison: read vs search vs write

Data structure	Read (by index / top)	Search (by value / key)	Write (insert / delete)	Best at
Array / dynamic array	O(1) fast	O(n), or O(log n) if sorted	O(n), except append / pop at end	Read
Linked list	O(n)	O(n)	O(1) at ends or with node handle	Write at known positions
Stack / queue / deque	O(1) at ends	O(n)	O(1) at ends	Ordered processing (LIFO / FIFO)
Hash map	O(1) by key	O(1)	O(1)	Search + write
Balanced BST	O(log n)	O(log n)	O(log n)	Ordered search + write
Heap	O(1) top	O(n)	O(log n)	Priority access

Data structure	Read (by index / top)	Search (by value / key)	Write (insert / delete)	Best at
Trie	O(L)	O(L) prefix	O(L)	Prefix search
B+ tree	O(log_B n)	O(log_B n) + fast range scan	O(log_B n)	Disk-based read + range

Decision shortcut

1. Need order or range / floor / ceiling? Balanced BST (or sorted array if read-only).
2. Need only lookup by key? Hash map.
3. Need only the min or max repeatedly? Heap.
4. Need FIFO or LIFO? Queue / stack / deque.
5. Need index access? Array / dynamic array.
6. Need prefix matching? Trie.
7. Need constant-time insert / delete in the middle with a handle? Doubly linked list (usually with a hash map to hold the handles).
8. Need grouping / connectivity? Union-Find.
9. Need range aggregates with updates? Segment tree / Fenwick tree.

Practical notes

- Hash map worst case O(n) is rare with a good hash function; in interviews state “O(1) average”.
- Dynamic array append is O(1) amortized; a single append that triggers a resize is O(n).
- Linked lists often lose to arrays in practice because of cache misses, even where the Big-O looks equal or better.
- Heap `build` from n items is O(n), not O(n log n); n separate pushes is O(n log n).
- Sorting a list once (O(n log n)) then binary searching beats repeated linear scans when searches outnumber about log n.
- Unbalanced BST degrades to a linked list on sorted input; use a balanced one (TreeMap) in production.